JS Functions

Shoaib Ghachi

MCT | Technical Trainer | Mentor |SME

Overview

Functions in JavaScript

-Function Declaration

-Function Expression

-Arrow Function

-Return Statement

-Function Callback

Functions in JavaScript

In JavaScript, functions are reusable blocks of code that can be called or invoked to perform a specific task. They encapsulate a set of instructions and can accept input parameters and return values. Here's the syntax for creating and using functions in JavaScript:

- **Function Declaration:**

A function declaration defines a named function that can be called later in the code.

```
function functionName(parameter1, parameter2) {
  // Code to be executed
}
```

```
// Function declaration
function greet(name) {
  return `Hello, ${name}!`;
}
```

```
// Function call
const greeting = greet("John");
console.log(greeting); // Output: "Hello, John!"
```

- **Function Expression:**

A function expression defines a function as part of a variable assignment.

```
var functionName = function(parameter1, parameter2) {
  // Code to be executed
};
```

functionName is the name of the variable holding the function.

parameter1 and parameter2 are optional parameters that can be passed to the function.

```
// Function expression
const greet = function(name) {
  return `Hello, ${name}!`;
};
```

```
// Function call
const greeting = greet("John");
console.log(greeting); // Output: "Hello, John!“
```

- **Arrow Function:**

Arrow functions provide a more concise syntax for creating functions, introduced in ECMAScript 6.

```
var functionName = (parameter1, parameter2) => {
  // Code to be executed
};
```

functionName is the name of the variable holding the function.

parameter1 and parameter2 are optional parameters that can be passed to the function.

```
// Regular function expression
const add = function(a, b) {
  return a + b;
};
console.log(add(2, 3)); // Output: 5

// Arrow function
const addArrow = (a, b) => a + b;
console.log(addArrow(2, 3)); // Output: 5
```

- **Return Statement:**

Functions can return a value by using the return statement, which ends the function execution and specifies the value to be returned.

```
function functionName() {
  // Code to be executed
  return value;
}
```

value is the value that the function returns. If omitted, the function returns undefined.

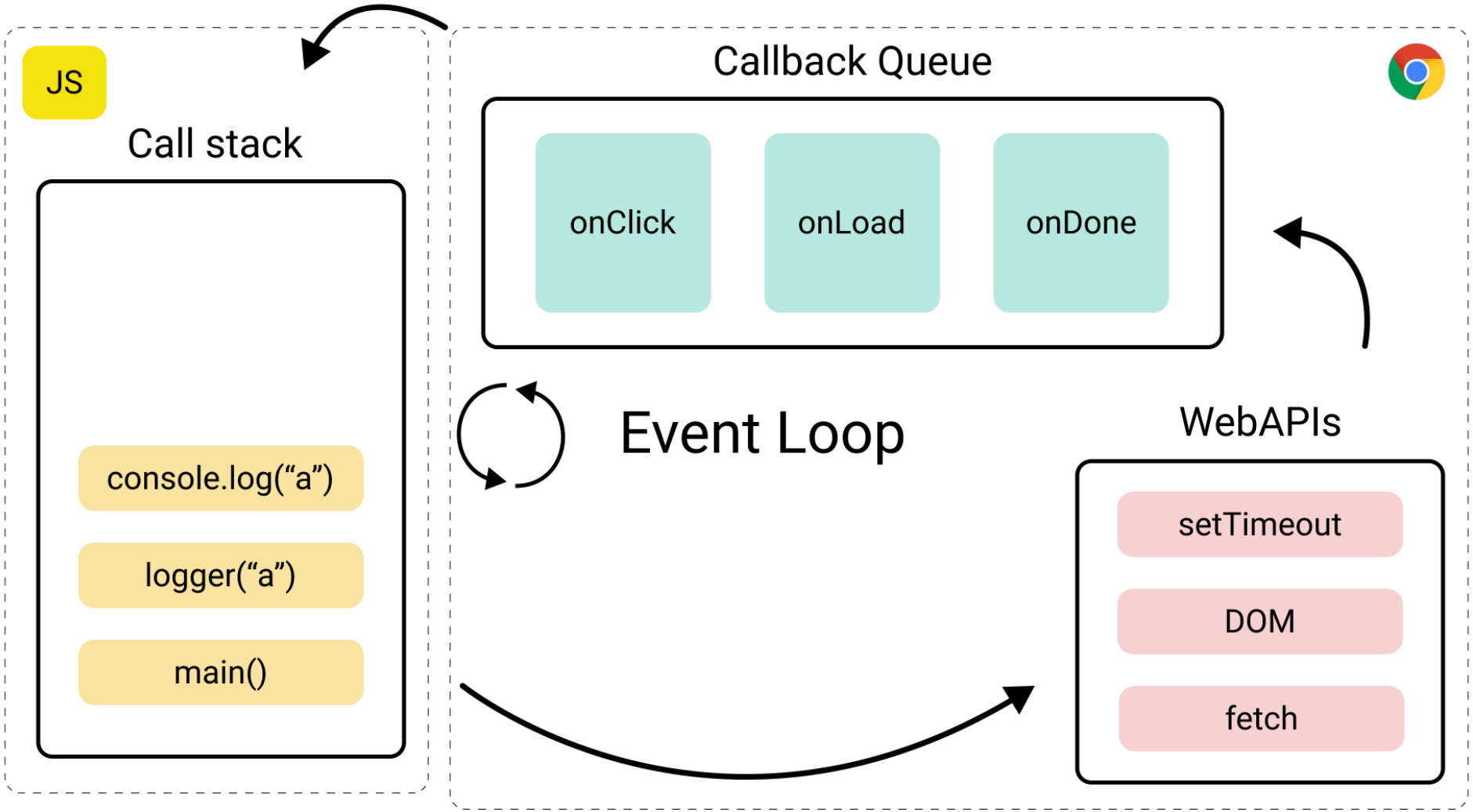
```
function greet(name) {
  if (!name) {
    return "Hello, stranger!"; // If name is falsy, return this value and exit the function
  }

  // This code will not be executed if the name is falsy
  return `Hello, ${name}!`;
}
console.log(greet("John")); // Output: "Hello, John!"
console.log(greet()); // Output: "Hello, stranger!"
```

- **Function Call Backs:**

A callback in JavaScript is a function that is passed as an argument to another function, and it is executed later, usually after some operation is completed.

```
function mainFunction(callback) {
  // Code to be executed before the callback
  // Call the callback function
  callback();
}

function callbackFunction() {
  // Code to be executed as the callback
}

// Call the main function and pass the callback function as an argument
mainFunction(callbackFunction);
```

```
function greet(name, callback) {
console.log("Hello " + name);
callback();
}

function sayBye() {
console.log("Goodbye!");
}

greet("Shoaib", sayBye);
```


JS
Call stack
console.log("a")
logger("a")
main()
Callback Queue
onClick
onLoad
onDone
Event Loop
WebAPIs
setTimeout
DOM
fetch

References

https://www.w3schools.com/js/js_functions.asp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions

https://www.javatpoint.com/javascript-function

https://www.geeksforgeeks.org/functions-in-javascript/

https://www.tutorialspoint.com/javascript/javascript_functions.htm

https://www.tutorialsteacher.com/javascript/javascript-function

https://www.javascripttutorial.net/javascript-function/